Qwen3.5_Env 高性能 P集群专属环境使用指南

📖 Qwen 3.5 / vLLM 高性能 P集群专属环境使用指南

简介：为了解决P集群底层系统版本过老导致的各种报错，我们已经提前在公共目录下为大家编译并打包好了一个“满血版”的 Qwen 3.5 / Qwen 3 运行环境（包含定制版的 PyTorch 2.7.1, vLLM 0.16.1, CUDA12.4, Flash-Attention 以及 MS-SWIFT 微调框架）。

大家**无需自己配置或编译任何底层库**，只需按照以下 4 个步骤解压激活，即可开箱即用。

环境路径： `/mnt/phwfile/datafrontier/share/qwen35_env.tar.gz`

🚀 快速启动流程

第一步：创建本地环境目录

在你的个人账号下，创建一个用于存放该环境的文件夹（建议直接放在 miniconda3 的 envs 目录下）：

代码块

```
mkdir -p ~/miniconda3/envs/qwen35_env
```

第二步：从公共区域解压环境

将打包好的环境压缩包解压到你刚刚创建的目录中。文件较大（包含完整的 CUDA 依赖和编译好的 C++ 算子），解压可能需要几分钟，请耐心等待：

代码块

```
tar -xzf /mnt/phwfile/datafrontier/share/qwen35_env.tar.gz -C
~/miniconda3/envs/qwen35_env
```

第三步：首次激活环境

此时环境尚未注册到你的 Conda 列表中，第一次激活必须使用绝对路径：

代码块

```
source ~/miniconda3/envs/qwen35_env/bin/activate
```

第四步：修复环境路径（核心且必须！）

这一步非常关键！ 由于环境是从构建者的目录打包过来的，内部包含了大量硬编码的绝对路径（如 Python 解释器路径）。你需要运行以下命令，系统会自动将这些路径替换为你自己的真实路径：

代码块

```
conda-unpack
```

（恭喜！执行完这一步，这个环境就彻底部署在你的账号下了。这个命令只需在第一次解压后运行一次。）

🧪 vLLM 性能与正确性测试（验证安装）

为了确保你的环境已完美适配当前节点的 GPU，并且核心加速算子（Flash-Attention、CUDA Graphs）均正常工作，请务必运行一次完整的**测试脚本**。

直接在**计算节点**的终端复制并粘贴以下整段代码并回车执行：

代码块

```
# 1. 注入防冲突环境变量（必填！防止 PyTorch 与 vLLM 编译冲突）
export TORCH_COMPILE_DISABLE=1
export TORCHDYNAMO_SUPPRESS_ERRORS=1
export VLLM_TORCH_COMPILE_LEVEL=0

# 2. 运行带测速的推理脚本
python - << 'EOF'
import time
from vllm import LLM, SamplingParams

# 使用极小模型快速验证环境
model_name = "facebook/opt-125m"

print(f"\n🚀 [测试开始] 正在拉起 vLLM 引擎，准备加载模型: {model_name} ...")
try:
    llm = LLM(model=model_name, trust_remote_code=True)

    prompts = [
        "What is the capital of China?",
        "Explain the concept of AI in one sentence."
    ]
    sampling_params = SamplingParams(temperature=0.7, max_tokens=50)

    print("\n⚡ 正在执行推理（请观察上方日志是否出现 'Capturing CUDA graphs'）...")
```

```
    start = time.perf_counter()
    outputs = llm.generate(prompts, sampling_params)
    end = time.perf_counter()

    for output in outputs:
        print(f"Result: {output.outputs[0].text.strip()}\n" + "-"*40)

    throughput = sum(len(o.outputs[0].token_ids) for o in outputs) / (end - start)
    print(f"\n✅ 环境验证成功！")
    print(f"📊 吞吐量: {throughput:.2f} tokens/s")

except Exception as e:
    import traceback
    print(f"\n❌ 测试失败！报错信息如下: \n{e}")
    traceback.print_exc()
EOF
```

🔍 **如何判断测试成功？**

1. **不报错崩溃**：脚本能够完整跑完并输出模型的英文回答。
2. **满血性能标志**：在终端打印的运行日志中，如果你看到了类似 `Capturing CUDA graphs (mixed prefill-decode, PIECEWISE): 100%` 的进度条，说明你的 vLLM 是满血状态，CUDA 图捕获完全正常！*（注：脚本结尾如果出现 `EngineCore_DP0 died unexpectedly` 属于正常退出时的守护进程误报，可直接忽略。）*

⚠ 核心注意事项

1. 运行微调/推理的“必加”环境变量（防崩溃）

在使用 `swift infer --infer_backend vllm` 调用大模型，或者进行 **GRPO 强化学习微调**时，PyTorch 的原生 Dynamo 编译机制与 vLLM 自己的图编译会发生底层冲突。

在运行相关的 Python 推理或训练脚本前，请务必在终端执行以下三行代码将编译权交给 vLLM：

代码块

```
# 屏蔽 PyTorch 原生编译，保留 vLLM 纯净编译环境，保障绝对稳定性
export TORCH_COMPILE_DISABLE=1
export TORCHDYNAMO_SUPPRESS_ERRORS=1
export VLLM_TORCH_COMPILE_LEVEL=0

# 测试 vLLM 加载与推理
python -c "import vllm; print('vLLM Load Success!')"
```

(如果不加这三行，程序在启动时可能会直接报错崩溃！)

2. **关于后续的日常使用**

在完成初次的 `conda-unpack` 后，以后你每次登录服务器，直接使用标准的 Conda 命令激活即可：

代码块

```
conda activate ~/miniconda3/envs/qwen35_env
```

3. **硬件计算节点要求**

该环境内的核心加速算子（如 Flash-Attention 和 vLLM）是针对特定的 GPU 架构进行过极致编译优化的。请确保你申请的计算节点显卡型号与该环境编译时的显卡型号一致（当前环境基于 **A100 + CUDA12.4** 进行编译 `TORCH_CUDA_ARCH_LIST=8.0`），否则在运行模型时可能会遇到 `No kernel found` 等底层算力不匹配的报错。

---

*以下内容为环境**构建指南**，普通用户可以忽略*

📖 Qwen 3.5 / vLLM 高性能 P集群专属环境构建指南

由于P集群操作系统（OS）太老（通常表现为 `glibc` 版本过低），直接从 PyPI 下载的预编译 `.whl` 包无法在P集群的系统上运行。

采取的“**环境隔离与全源码编译**”策略：利用 Conda 安装一套现代化的编译器工具链，强行“欺骗”编译器和链接器，让它们在编译 Qwen 3.5 环境时使用 Conda 内部的高版本库，而不是系统自带的老旧库。

核心流程概览

阶段	步骤	目的
1. 基础设施	编译环境劫持 (Compiler Hijacking)	用 Conda 的 GCC 11+ 替代系统老旧的 GCC 4.8，解决 GLIBC 兼容问题。
2. 核心框架	PyTorch 2.7.1 源码编译	针对 CUDA 12.4 驱动和当前 CPU 架构进行深度定制安装。
3. 高性能算子	Triton & Flash-Attn 编译	Qwen 3.5 的核心加速组件，必须源码编译以匹配 PyTorch。
4. 逻辑修复	Transformers RoPE Hotfix	修复特定版本中 RoPE 旋转位置编码的 Bug，防止模型输出乱码。
5. 推理引擎	vLLM 0.17.0 深度编译	锁定编译器路径，确保推理引擎能正确调用 CUDA Graph 提高吞
6. 框架适配	Swift / FLA / Causal-Conv1d	安装微调框架和特定的线性注意力算子，完成环境闭环。

第一部分：环境编译构建 详细执行文档

第一阶段：构建 Conda 内部编译器工具链

目的： 在 Conda 环境内安装一套高版本的 GCC 和开发工具，使其完全脱离对系统老旧 `glibc` 和 `gcc` 的依赖。

代码块

```
# 1. 进入你的 Conda 环境
conda activate qwen_env

# 2. 安装编译器与核心开发库# 指定 sysroot 2.17 是为了确保在老旧系统上的最大兼容性
conda install -y -c conda-forge \
    gcc_linux-64 \
    gxx_linux-64 \
    gfortran_linux-64 \
    sysroot_linux-64=2.17 \
    cmake \
    make \
    binutils

# 3. 验证安装 (确认路径在你的 conda env 目录下)
which x86_64-conda-linux-gnu-gcc
```

第二阶段：准备编译器环境（环境劫持）

目的： 通过软链接和环境变量，强制后续所有的 `pip` 和 `setup.py` 指向我们刚安装的高版本编译器。

代码块

```
# 创建编译器软链接目录，防止系统路径干扰
mkdir -p $CONDA_PREFIX/bin/compiler_links
cd $CONDA_PREFIX/bin/compiler_links
ln -sf ../x86_64-conda-linux-gnu-gcc gcc
ln -sf ../x86_64-conda-linux-gnu-g++ g++
ln -sf ../x86_64-conda-linux-gnu-c++ c++
cd -

# 注入环境变量（建议在本次 Session 中一直保持）
export PATH="$CONDA_PREFIX/bin/compiler_links:$PATH"
```

```
export CC="$CONDA_PREFIX/bin/compiler_links/gcc"
export CXX="$CONDA_PREFIX/bin/compiler_links/g++"
export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:$LD_LIBRARY_PATH" # 告诉系统动态链接库在哪里
export LDFLAGS="-L$CONDA_PREFIX/lib -Wl,-rpath,$CONDA_PREFIX/lib" # 告诉 C++ 链接器（ld）去哪里找库，并且把路径写死在生成的二进制文件里（rpath）
```

第三阶段：编译 PyTorch 与 TorchVision

目的： 构建底层张量计算框架。

代码块

```
# 1. 编译 PyTorch==2.7.1 (假设你已经在 pytorch 源码目录下)# 注意：确保已经 git checkout 到对应版本分支 (torch2.7.1)
python setup.py install

# 2. 安装 Torchvision==0.22.1 及其依赖
conda install scipy -y
pip install "setuptools<70.0.0"
conda install -y -c conda-forge libjpeg-turbo libpng
FORCE_CUDA=1 pip install --no-build-isolation git+https://github.com/pytorch/vision.git@v0.22.1
```

第四阶段：Transformers 修复与 Triton 编译

代码块

```
# 1. 安装并修复 Transformers (Qwen 3.5 必须修复 RoPE)
pip install transformers==5.2.0 -U --no-deps
# 应用 RoPE 热补丁
python - <<'PY'
import pathlib
import transformers.modeling_rope_utils as m
p = pathlib.Path(m.__file__)
text = p.read_text(encoding="utf-8")
old = 'ignore_keys_at_rope_validation = ignore_keys_at_rope_validation | {"partial_rotary_factor"}'
new = 'ignore_keys_at_rope_validation = set(ignore_keys_at_rope_validation) | {"partial_rotary_factor"}'if old in text and new not in text:
    p.write_text(text.replace(old, new), encoding="utf-8")
print(f"[Info] rope hotfix applied to: {p}")
```

```
PY

# 2. 源码编译 Triton==3.6.0
git clone https://github.com/triton-lang/triton.git (注意3.6.0版本经测试可用)
cd triton
pip install -r python/requirements.txt
# 🛡️ 防御机制：祭出我们之前配好的 GCC 11（千万别忘，否则又会被系统的老 GCC 搞崩溃）
export PATH="$CONDA_PREFIX/bin/compiler_links:$PATH"
export CC="$CONDA_PREFIX/bin/compiler_links/gcc"
export CXX="$CONDA_PREFIX/bin/compiler_links/g++"
# 开始正式编译
pip install -v .
cd ..
```

第五阶段：Flash-Attention 与算子库编译

目的： 安装核心加速算子。**注意：** `TORCH_CUDA_ARCH_LIST` A100 设为 `8.0`，H100 设为 `9.0`，L40S/4090 设为 `8.9`

代码块

```
# 1. 编译 FLA (Flash Linear Attention)
pip install --no-binary :all: --no-cache-dir git+https://github.com/fla-org/flash-linear-attention.git

# 2. 编译 Flash-Attn
# 【关键】配置环境变量
export PATH="$CONDA_PREFIX/bin/compiler_links:$PATH"
export CC="$CONDA_PREFIX/bin/compiler_links/gcc"
export CXX="$CONDA_PREFIX/bin/compiler_links/g++"
#【关键】将临时目录设为计算节点的本地内存盘，速度最快且没冲突
export TMPDIR=/dev/shm
export FLASH_ATTENTION_FORCE_BUILD=TRUE
#【关键】限制并行任务数为 4 或 8，防止压垮文件系统
export MAX_JOBS=4
#【关键】只编译特定架构（以 8.0 为例），避免漫长的全架构编译，如果是 H100 请改为 9.0
export TORCH_CUDA_ARCH_LIST="8.0"  # 此处根据实际显卡调整
# 开始编译安装
pip install flash-attn --no-cache-dir --no-build-isolation

# 3. 编译 Causal-Conv1d
git clone https://github.com/Dao-AILab/causal-conv1d.git
cd causal-conv1d
CAUSAL_CONV1D_FORCE_BUILD=TRUE pip install . --no-build-isolation --no-cache-dir
```

```
cd ..
```

第六阶段：MS-SWIFT 微调框架安装

目的： 提供 Qwen 系列模型的训练和微调入口。

代码块

```
# 1. 安装核心依赖
conda install -y -c conda-forge pyarrow av
pip install tiktoken --only-binary :all:
pip install decord --only-binary :all:
pip install wandb deepspeed ninja --only-binary :all:

# 2. 安装 Swift 与 Qwen-VL 工具
pip install git+https://github.com/modelscope/ms-swift.git # 4.1.0
pip install "qwen_vl_utils>=0.0.14" -U
```

第七阶段：vLLM 深度编译

这是最困难的一步，必须强制 CMake 使用 Conda 编译器。

目的： 极致的推理加速，开启 CUDA Graph 满血性能。

代码块

```
# 1. 下载并配置 vLLM
git clone https://github.com/vllm-project/vllm.git # 0.17.0
cd vllm

# 2. 锁定 CMake 和 NVCC 的编译器注入
export PATH="$CONDA_PREFIX/bin/compiler_links:$PATH"
export CC="$CONDA_PREFIX/bin/compiler_links/gcc"
export CXX="$CONDA_PREFIX/bin/compiler_links/g++"
export CMAKE_ARGS="-DCMAKE_C_COMPILER=$CC -DCMAKE_CXX_COMPILER=$CXX -DCMAKE_CUDA_HOST_COMPILER=$CXX"
export NVCC_PREPEND_FLAGS="-ccbin $CXX"

export MAX_JOBS=4
export VLLM_MAX_JOBS=4
export CMAKE_BUILD_PARALLEL_LEVEL=4

# 3. 安装依赖并编译
```

```
pip install -r requirements/cuda.txt
pip install setuptools_scm Cython pybind11
# 开始最终编译安装，加上 --no-build-isolation 确保使用刚才亲自编译的 PyTorch 2.7
pip install . --no-build-isolation --no-deps -v 2>&1 | tee build_vllm.log

# 4. 补齐 vLLM 运行时依赖
pip install cloudpickle gguf cachetools cbor2 openai_harmony llguidance xgrammar partial_json_parser blake3
export CFLAGS="-fPIC" CXXFLAGS="-fPIC" LDFLAGS="-fno-lto"
# 重新尝试安装 xgrammar (如果上一步安装成功则不需要)
pip install xgrammar --no-cache-dir
cd ..
```

第八阶段：vLLM 性能与正确性测试

目的： 验证环境是否真正支持 vLLM 的满血推理。

代码块

```
# 环境变量屏蔽部分可能冲突的编译优化
export TORCH_COMPILE_DISABLE=1
export TORCHDYNAMO_SUPPRESS_ERRORS=1
export VLLM_TORCH_COMPILE_LEVEL=0

python - << 'EOF'
import time
from vllm import LLM, SamplingParams

# 使用较小的模型进行验证
model_name = "facebook/opt-125m"
print(f"\n🚀 [测试开始] 正在加载 vLLM 引擎...")

try:
    # 允许 vLLM 尝试捕获 CUDA Graphs
    llm = LLM(model=model_name, trust_remote_code=True)

    prompts = [
        "What is the capital of China?",
        "Explain the concept of AI in one sentence."
    ]
    sampling_params = SamplingParams(temperature=0.7, max_tokens=50)

    print("\n⚡ 正在执行推理 (观察日志是否出现 Capturing CUDA graphs)...")
    start = time.perf_counter()
    outputs = llm.generate(prompts, sampling_params)
```

```
    end = time.perf_counter()

    for output in outputs:
        print(f"Result: {output.outputs[0].text.strip()}")

    throughput = sum(len(o.outputs[0].token_ids) for o in outputs) / (end - start)
    print(f"\n✅ 推理成功! 吞吐量: {throughput:.2f} tokens/s")

except Exception as e:
    print(f"\n❌ 测试失败: {e}")
EOF

```

第二部分：集群公共目录打包与分发

制作自动激活脚本并打包

目的： 让其他人无需编译，直接解压使用。

代码块

```
# 1. 预埋自动环境变量 Hook（使用者激活环境时自动赋予 C++ 库查找路径）
mkdir -p $CONDA_PREFIX/etc/conda/activate.d
mkdir -p $CONDA_PREFIX/etc/conda/deactivate.d

cat << 'EOF' > $CONDA_PREFIX/etc/conda/activate.d/env_vars.sh
#!/bin/bashexport OLD_LD_LIBRARY_PATH="$LD_LIBRARY_PATH"export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:$LD_LIBRARY_PATH"export PATH="$CONDA_PREFIX/bin/compiler_links:$PATH"
EOF

cat << 'EOF' > $CONDA_PREFIX/etc/conda/deactivate.d/env_vars.sh
#!/bin/bashexport LD_LIBRARY_PATH="$OLD_LD_LIBRARY_PATH"unset OLD_LD_LIBRARY_PATH
EOF

# 2. 清理巨大的编译缓存（可选，为打包瘦身）
conda clean --all -y

# 3. 将环境打包并放到集群公共区域
conda install -c conda-forge conda-pack -y
```

```
conda pack -n qwen35_env -o /mnt/phwfile/datafrontier/share/qwen35_env.tar.gz --ignore-missing-files
```

第三部分：避坑指南总结

1. **编译失败怎么办？** 如果报错提示 `virtual memory exhausted`，说明 `MAX_JOBS=4` 太高，请将其改为 `export MAX_JOBS=1` 重试。
2. **验证环境：** 编译完后，一定要运行脚本最后的 `facebook/opt-125m` 测试。如果能看到 `Capturing CUDA graphs`，说明你的 vLLM 是“满血版”，否则性能会大打折扣。
3. **持久化配置：** 如果下次登录服务器发现 `import torch` 报错，记得重新执行第一阶段的 `export LD_LIBRARY_PATH` 那几行代码。